

Polygraph: Automated False-Positive Classification of SAST Findings

Alexander Braml
Chair of Computer Engineering
University of Passau
Passau, Germany
alexander.braml@uni-passau.de

Florian Frank
Chair of Computer Engineering
University of Passau
Passau, Germany
florian.frank@uni-passau.de

Felix Klement
Chair of Computer Engineering
University of Passau
Passau, Germany
felix.klement@uni-passau.de

Abstract—Static application security testing (SAST) tools frequently generate findings that, upon further investigation, turn out to be false positives (FPs) rather than genuine vulnerabilities. Triageing these FPs by hand is slow and draws effort away from the real weaknesses that demand attention. In this work, we present Polygraph, a tool that classifies each SAST finding as a true positive (TP) or an FP. It reads and writes reports in standard static analysis results interchange format (SARIF) format and operates between any compliant scanner and viewer. For each finding, it assembles a compact, finding-specific context from the surrounding code, together with a data-flow analysis that traces how input reaches the affected location. It then queries an Large Language Model (LLM) for a verdict. When this static context does not settle a finding, Polygraph escalates it to an autonomous agent that explores the repository directly. Evaluated with four open-weight LLMs on the synthetic OWASP Benchmark for Python, Polygraph suppresses 95.8% of its FP traps at a suppression precision of 0.997 and a dangerous-suppression rate of 0.4%, for an F1-score of 0.977 and a balanced accuracy of 0.973. On RealVuln, a corpus of real-world vulnerable applications whose findings depend on code spread across a repository rather than a single file, it reaches a lower F1-score of 0.797 at a balanced accuracy of 0.880, suppressing 82.5% of its traps at a dangerous-suppression rate of 5.0%. Since these results hold with locally-hosted, open-weight models that keep code from leaving an organization’s own infrastructure, Polygraph makes a reliable source of SAST triage decisions available even in security-sensitive settings that cannot rely on third-party inference providers.

Index Terms—Static Application Security Testing, SAST, False-Positive Reduction, Software Security

I. INTRODUCTION

With the rise of agentic development workflows, source code is produced at an unprecedented speed, resulting in faster release cycles, while simultaneously increasing the manual code review load. Code produced by coding agents frequently contains exploitable weaknesses even when functionally correct [1], and manual security assessment, adding to the burden of functional code review, does not scale with the rate at which such code is produced. SAST therefore serves as an increasingly important safeguard, inspecting source code without executing it to find potential vulnerabilities before they reach production.

A recurring problem is that a large share of reported findings does not correspond to a real vulnerability in its specific context. These FPs carry a measurable cost. According to a

2025 industry report, 63% of security teams spend at least four hours per week triaging FPs [2]. Time spent dismissing false alerts is time not spent on real vulnerabilities, and a steady stream of noise erodes developer trust in the tools themselves [3].

Reducing this burden remains difficult by conventional means. Whether a given finding is an FP can rarely be determined from the finding alone. It also depends on things like how the surrounding code uses the flagged construct or whether user input can reach it without being sanitized first. A static scanner reports the finding because it cannot resolve this context on its own, so a deterministic filter layered on top would have to reconstruct this semantic understanding the scanner lacks. An LLM can reason over this kind of code context, which makes it a suitable basis for this task.

This paper presents Polygraph, a tool that classifies SAST findings as TPs or FPs. This reduces the effort necessary to deal with large volumes of SAST findings and allows developers to focus on genuine vulnerabilities, directly solving the problems conventional scanners introduce.

Polygraph reads and writes standard SARIF reports, so it sits between any compliant scanner and viewer. It parses the heterogeneous set of findings, de-duplicates them based on the type of the finding and location, and categorizes them into Common Weakness Enumeration (CWE) classes. Afterwards, it assembles finding-specific contexts that include the surrounding code, documentation and on-demand data-flow analysis, and queries an LLM for a verdict. When the static context does not suffice for a decision, the model declines to classify, and Polygraph escalates the finding to an autonomous agent that explores the repository directly.

On the OWASP Benchmark for Python [4], Polygraph reaches an F1-score of 0.977 on the FP class at a balanced accuracy of 0.973, suppressing 95.8% of the benchmark’s FP traps at a suppression precision of 0.997 and a dangerous-suppression rate of only 0.4%. RealVuln [5] is a benchmark of intentionally vulnerable open-source applications whose findings, unlike OWASP’s single-file cases, depend on code distributed across a repository. It is correspondingly harder: Polygraph reaches a lower F1-score of 0.797 at a balanced accuracy of 0.880. It still suppresses 82.5% of the benchmark’s FP traps, at a dangerous-suppression rate of 5.0%.

In summary, this paper makes the following contributions:

- Polygraph, a SARIF-native tool that assembles a finding-specific context from surrounding code, documentation, and on-demand data-flow analysis, and queries an LLM to classify each finding as a TP or FP, so it drops into any pipeline between an existing scanner and viewer.
- A two-stage classification design in which findings the static context cannot settle are escalated to an autonomous agent that explores the repository directly, keeping the common case cheap while still resolving the harder, context-dependent cases.
- An evaluation on the OWASP Benchmark for Python and on RealVuln, a repository-scale benchmark of intentionally vulnerable open-source applications, showing that Polygraph suppresses 95.8% and 82.5% of each benchmark’s FP traps, respectively, while keeping the rate of dangerously suppressed TPs at 0.4% and 5.0%.

A. Background

SAST tools statically analyze a program’s source or compiled code and report code patterns that may indicate a security vulnerability, such as an injection flaw or unsafe handling of user input [6]. Each issue such a tool reports is a finding, annotated with a rule identifier, a severity, and the code location it concerns. Because these tools only statically analyze the source code and do not interpret the context around the finding, like sanitization steps or dynamic behavior at runtime, they also report many findings that are not real vulnerabilities in their context. Some tools also over-approximate and rather report an FP than missing a TP, which results in further increases of the overall volume of findings.

Individual SAST tools historically output their results in incompatible, proprietary formats, which complicates aggregating and processing findings across tools. SARIF addresses this as an OASIS standard that defines a single JSON-based format for static-analysis output [7]. A SARIF report records each finding together with its rule, severity, and location, and it provides standard fields such as `Suppressions` to mark findings that a viewer should hide. Polygraph reads SARIF as input and writes SARIF as output, which lets it operate between existing scanners and any viewer or dashboard that supports SARIF.

Many SAST findings deal with uncontrolled and unsanitized user-input reaching a sensible sink, such as a database query or system command. For these findings, the way data flows into the vulnerable line is essential to determine if it is genuine. Data-flow analysis addresses this by statically tracing values from their source, where the data enters the program, to the sink, which in our case is the location reported by the SAST tool. A common form is taint analysis, which tracks whether data from an untrusted source, like user input, reaches a security-sensitive sink, such as a database query, and whether a sanitizer neutralizes the data along the way [8]. Polygraph computes this information using CodeQL [9], a well-established framework for semantic code analysis that

models a program as a queryable database and includes data-flow and taint-tracking libraries.

II. RELATED WORK

ZeroFalse and QASecClaw are the two systems most closely related to Polygraph. Both combine a SAST scanner with an LLM that reviews each finding to decide whether it is a real vulnerability, the same basis Polygraph builds on, and both decide it in a single pass. Xiong and Zhang instead evaluate autonomous agents that investigate every finding on their own, which is the second stage Polygraph applies selectively when necessary.

ZeroFalse [10] operates only on findings from CodeQL. It treats such alerts as structured contracts and enriches them with the scanner’s own data-flow trace, the surrounding code, and information specific to each CWE class before a single LLM pass returns a verdict. Across the OWASP Java Benchmark and a curated real-world dataset it reaches F1-scores of 0.912 and 0.955, below Polygraph’s 0.977 on the Python variant of the OWASP Benchmark. Its design is based on CWE-specific tailoring, and the authors name contextual blindness, the difficulty of supplying sufficient context for a multi-file finding, as an open problem. Polygraph addresses this problem with the autonomous escalation agent that gathers evidence on-demand. It also maintains no prompt per weakness class, but uses a generic prompt template for all findings that includes a description of the concrete finding. Furthermore, rather than reuse the data-flow traces the scanner potentially emitted, Polygraph computes its own taint analysis on-demand, which removes the dependence on a scanner that populates SARIF flow traces comprehensively. ZeroFalse depends on CodeQL for exactly this reason. As a result, it cannot triage findings of arbitrary scanners, while Polygraph sits between any SARIF-compatible scanner and viewer.

QASecClaw [11] orchestrates several role-specific agents that cover scan planning, Semgrep-based scanning, evidence correlation, and an LLM SAST Filter Agent. It classifies each finding from its enclosing source file and CWE label, with a fail-open policy that retains a finding whenever the model fails. On OWASP Benchmark v1.2 it achieves an F1-score of 0.909 and removes 88.6% of FPs. Its filter, however, performs no data-flow analysis and reasons only over the enclosing file. Its agents also form a fixed pipeline rather than an investigator that gathers missing evidence when needed. In contrast, on the Python variant of the OWASP Benchmark, Polygraph reaches an F1-score of 0.977 while suppressing 95.8% of the FP traps.

Xiong and Zhang [12] evaluate three general-purpose coding agents, Aider, OpenHands, and SWE-agent, as FP filters under three different models, rather than proposing a system of their own. Each agent receives the alert and the source code and settles the finding on its own. On OWASP Benchmark v1.2 for Java they reduce an initial FP rate of 98.3% to 6.3%, and the result carries over to C/C++ findings.

ZeroFalse and QASecClaw judge every finding in a single, fixed-context pass, and neither recovers when the evidence a verdict needs is missing from that context. Xiong and Zhang

measure what this costs. A baseline that receives exactly the files their agent had read, but answers in one pass, reaches an F1-score of 0.516 against the agent’s 0.955, so the deciding factor is the iterative gathering of evidence rather than a larger context. Polygraph therefore treats uncertain as a first-class verdict. The cheap first stage answers the findings it can, and only those it cannot resolve escalate to an autonomous agent that explores the source code. This confines expensive exploration to the hard-to-decide cases that the two prior systems leave unresolved, directly attacks the contextual-blindness limitation that ZeroFalse names, and costs far less than running an agent on every finding, at 8,196 tokens per OWASP finding on average against the 130,846 tokens per alert their best configuration needs.

The same design also decides how many real vulnerabilities survive. Their best configuration classifies 22.25% of the benchmark’s real vulnerabilities as FPs, which leads the authors to reject such agents for automatic suppression. Polygraph reaches a dangerous-suppression rate of 0.4% on the Python variant of the OWASP benchmark, a different corpus but the same weakness families, because its escalation trigger is asymmetric. It fires on every uncertain verdict and every FP below high certainty, but never on a TP, so suppression is the only verdict that has to survive a second pass.

Two further differences widen Polygraph’s applicability. All three evaluations cover Java, ZeroFalse through CodeQL, QASecClaw through Semgrep, and Xiong and Zhang through four scanners at once and a small C/C++ sample. In contrast, Polygraph supports Python, C, C++, JavaScript, and TypeScript first-class, but can also handle findings in other languages. Where ZeroFalse consumes SARIF only from CodeQL and QASecClaw consumes Semgrep JSON, Polygraph both reads and writes regular SARIF, marking FPs through the standard `suppressions` field. This way, it slots between any compliant scanner and viewer. A prompt- and tool-hashing cache further makes repeated scans of an unchanged codebase effectively free, a concern that none of the three addresses.

On their respective OWASP evaluations, ZeroFalse and QASecClaw report F1-scores of 0.912 and 0.909, while Polygraph reaches 0.977. This gap has two causes. Polygraph suppresses more FPs and, at the same time, misclassifies fewer real vulnerabilities as FPs, so less noise reaches the reviewer without more real vulnerabilities being hidden. The agents of Xiong and Zhang achieve only the first, as they remove a comparable share of the noise but hide far more real vulnerabilities. Only suppression that is safe in both directions can be trusted, and that is what lets a team act on Polygraph’s verdicts and re-examine the escalated cases instead of double-checking the whole report.

III. APPROACH

Polygraph takes two inputs: a SARIF report that lists all findings and the root directory of the analyzed source code. From these, it classifies every finding as a TP or an FP in a two-stage process and writes the verdicts back into an annotated copy of the report.

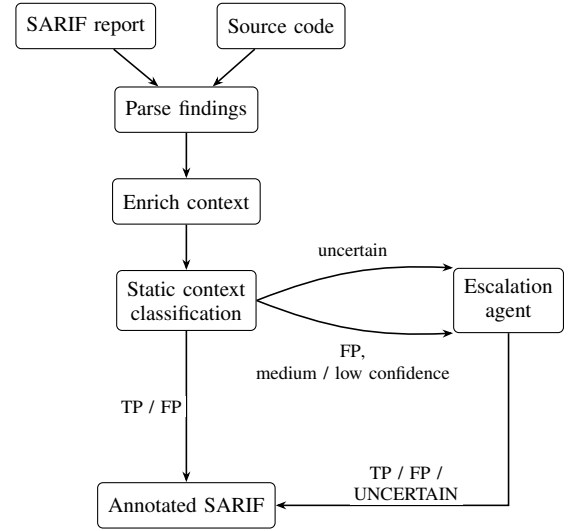


Fig. 1. Overview of the Polygraph pipeline. Findings parsed from the SARIF report are enriched with context from the source tree and classified by the first-stage LLM. Only findings it marks uncertain or FP with medium or low confidence escalate to the autonomous agent before Polygraph writes the annotated SARIF report.

A. Architectural Overview

The pipeline, shown in Fig. 1, first reads and parses the SARIF report and then iterates over all findings. For each finding, Polygraph assembles the required context, renders it into a prompt, and sends the prompt to the model for classification. Context generation is modular. A new enricher can extend the context without any change to the surrounding pipeline. Prompt templates are version-controlled and adjustable, as Polygraph inserts the context into templates rather than constructing them in code. Model endpoints are interchangeable as well, and any OpenAI-compatible endpoint is supported. When the model returns uncertain or FP with medium or low certainty for a finding, Polygraph passes that finding to the second stage using an autonomous agent.

To keep re-runs cheap and reproducible, Polygraph hashes every rendered prompt and, with an optional SQLite cache keyed by that hash and model name, skips the model call for identical prompts. For escalated findings, it hashes the initial prompt as well as the tool calls and their results, so even the results of these findings can be efficiently cached. Because context assembly and tool calling are deterministic, re-scanning an unchanged codebase is effectively free, and across successive scans only the few newly surfaced findings require classification, while unchanged findings reuse their earlier verdicts.

Polygraph finally writes a copy of the original SARIF report, enriched with additional attributes. For an FP it adds the standard SARIF `suppressions` field and records the reasoning behind the decision. For every finding it additionally adds a custom property that holds the verdict and the reasoning. A dashboard or SARIF viewer can read these attributes to display the verdicts, which lets developers concentrate on the TPs.

A central design goal was to keep Polygraph extensible to further languages. By default, Polygraph supports every language and technology in a basic way with line-based snippet extraction and no data-flow analysis, while the remaining steps operate unchanged. Full support of the additional snippet-extraction strategies requires updating the tree-sitter integration to install and import the grammar for the new language. Data-flow analysis requires a new custom query, because CodeQL, the framework we use for data-flow analysis, is language-agnostic in principle but still needs language-specific adjustments. Polygraph currently fully supports Python, C, C++, JavaScript, and TypeScript.

Every stage of the pipeline fails safely. If a finding cannot be classified, because context assembly fails or because the model request errors out, Polygraph keeps it in the resulting report without annotation. A failure therefore never suppresses a finding, so the report remains trustworthy even when parts of the pipeline do not succeed.

B. Stage 1: Static Classification

The first step in classifying a finding is to assemble the context for the LLM to use to output a verdict. This context should be enough to settle a verdict, but, to not confuse the model with too much context, it should be compact. To dynamically generate this context and test different configurations, Polygraph relies on Enrichers, which form the foundation of context generation, and it currently implements three of them. The `HeaderEnricher` extracts the header comment from the first lines of a file, following the comment syntax of the file’s language. The file header often states what the file does, who wrote it and why, which can help the model judge a finding in that file. The `SnippetEnricher` extracts the code that surrounds the finding. It parses the file with tree-sitter and offers several strategies to extract code context. One can include the enclosing method, class or file or a configurable number of lines above and below the affected line. The immediately surrounding code is essential for the model to understand what the code does and does not do. A possible SQL injection for example needs to know whether sanitization occurs before user-input reaches a query, which can be checked using the code surrounding the finding.

The `DataflowEnricher` performs data-flow analysis on all variables on the affected line and shows how data reaches those variables.

Because some tools also scan the git history, all enrichers read the affected file at the exact commit of the finding to get the correct file content. Each enricher also carries a numeric priority, so that a configurable token limit drops low-priority context first. The output of the enrichers is then assembled into a user prompt describing the concrete finding that is to be classified. The system prompt is static and does not change between findings. It gives instructions on how to classify findings and frames the classification as one question: does the finding’s described weakness hold in the shown code? It also provides detailed criteria for tracing sources to sinks and confirming mitigation. The prompt also explicitly

steers the model towards uncertain whenever the context is not enough to settle a finding. This system prompt together with the customized user prompt is then sent to the LLM for classification. The model then returns structured output containing a verdict, reasoning and a certainty indicating how confident the LLM is in its answer.

If the model is sure this is an FP or TP, this verdict is annotated in the resulting SARIF-report. If the model returns uncertain or FP with only a medium or lower certainty, the second stage using the autonomous escalation agent is activated. Activating it also for low- and medium-confidence FPs as well ensures the model checks this verdict once more with a more-capable model, as suppressing real TPs is a costly mistake.

C. Stage 2: Autonomous Classification Agent

Findings that the first stage does not settle are handed to an autonomous escalation agent that has access to various read-only tools to interact with the source code. The agent investigates the finding on its own and must commit to the same structured verdict the first stage returns.

It has access to three read-only tools. `read_file` returns a source file with line numbers, either a requested range or a truncated version of the file together with information about its size. `list_directory` lists the entries of a directory, and `search_files` searches the repository for a regular expression and returns the matching lines with their locations. Every path a tool receives is resolved against the repository root and rejected if it escapes it, and each tool truncates its output so that a single call cannot flood the context window. A configurable budget of model requests bounds the investigation. If the agent exhausts it, Polygraph reports uncertain, leaving the annotated finding in the report for manual review.

The agent deliberately does not receive the static context of the first stage. Its user prompt includes only data about the finding as well as the first-stage reasoning, which names the missing evidence. It also prescribes an “investigate then commit” pattern, where the agent should first gather comprehensive evidence before committing to a verdict. The system prompt on the other hand includes the agent’s role, the available tools and instructions on how to classify findings as well as a set of rules preventing shortcuts. These rules define things like guards only counting as genuine mitigation if their actual implementation was read or secrets that only count as real ones if they are consumed by security-sensitive code.

Caching this stage requires more effort than caching the first, because the verdict depends on files the prompt does not contain. Polygraph records every tool call of a run together with a hash of its result. On a later run it replays the recorded calls against the current source tree and reuses the cached verdict only if every result reproduces exactly and any deviation triggers a fresh investigation. These records double as an audit transcript of what the agent read. The agent’s model is configured independently of the first-stage model, so that the minority of findings reaching this stage can be resolved by a

stronger model and the extra cost arises only where it pays off.

IV. EVALUATION

This evaluation measures how accurately Polygraph classifies SAST findings as real or FPs, and how this accuracy varies across different LLMs. At its best, Polygraph reaches an F1-score of 0.977 and a suppression precision of 0.997. The evaluation rests on two open-source benchmarks. The OWASP Benchmark for Python [4] contains 1230 test cases, each labeled as real or not. RealVuln [5] is a collection of intentionally vulnerable open-source applications. We use its human-authored ground-truth, which comprises 748 findings over 236 files and 73 CWEs, 622 real vulnerabilities and 126 hand-authored traps that look vulnerable but are not. The distribution of findings in OWASP is skewed the other way: 452 real vulnerabilities against 778 traps. Both benchmarks grade detection rather than FP classification, so we synthesize one SARIF-report with one finding for each ground-truth entry. The details of these findings include a basic description, the CWE assigned as well as the location of the finding. The assembled report is then passed to Polygraph together with the source code. The source code is cleaned of any ground-truth data, documentation or comments that may hint towards the verdict of a finding. Otherwise the LLM may base its verdict on the ground-truth data, which would invalidate the evaluation results.

An FP verdict is the positive class for precision, recall, and the F1-score throughout, since detecting FPs is the goal. Plain accuracy says little, as the benchmarks are unbalanced in opposite directions. We therefore report the balanced accuracy and the Matthews Correlation Coefficient (MCC) together with the dangerous-suppression rate

$$\frac{\text{real vulnerabilities classified as FPs}}{\text{real vulnerabilities}},$$

which indicates how many real vulnerabilities were wrongly suppressed.

Every run uses method-level snippet extraction, the file header enricher, and no data-flow analysis, and the cache is disabled, so every finding is freshly classified and timing stays consistent across runs. Both stages use the same model, and escalation follows the default policy. The rendered prompts are identical across all runs.

Data-flow analysis is the slowest step in practice, taking several minutes even for simple projects, longer than the model call that follows it. Since its result is needed for every context and prompt assembly, enabling it renders the cache ineffective.

Four open-weight models classified both benchmarks under an identical prompt and context configuration, each serving both stages: Qwen 3.8 27B, Gemma 4 31B, Qwen 3.6 35B, and Qwen3-Next-80B. We chose these four as well-known, readily available models that span a wide range of parameter counts and two model families. These models can all be hosted locally, which was a deliberate choice. Security-sensitive findings may contain identifying information or sensitive source

TABLE I
VERDICTS OF THE QWEN 3.8 27B RUN OVER BOTH CORPORA. ROWS GIVE THE BENCHMARK LABEL, COLUMNS THE VERDICT POLYGRAPH ASSIGNED. UNCERTAIN VERDICTS (2 ON OWASP, 6 ON REALVULN) AND FINDINGS THAT ERRORED (7 AND 6) ARE COUNTED IN THE CLASS SIZES BUT OMITTED FROM THE COLUMNS.

Oracle	OWASP		RealVuln	
	Verdict FP	Verdict TP	Verdict FP	Verdict TP
Real vulnerability	2	447	31	581
Trap	745	27	104	20
Class size	452 / 778		622 / 126	

code, which is better confined to a trusted environment than sent to third-party inference providers. This also reflects deployment reality, as many organizations will not permit code or vulnerability data to leave their infrastructure [13], so a pipeline that depends on external APIs would be unusable in these settings.

A. Classification Quality

Table I shows a run over both benchmarks with Qwen 3.8 27B, the best-performing model in our tests. On the OWASP Benchmark, Polygraph suppresses 745 of 778 traps at a suppression precision of 0.997, for an F1-score of 0.977 at a balanced accuracy of 0.973 and an MCC of 0.950. Two of 452 real vulnerabilities were suppressed and 98.9% retained.

For RealVuln, suppression recall stays comparable at 0.825, but precision drops to 0.770, for an F1-score of 0.797 at a balanced accuracy of 0.880 and an MCC of 0.762. Here 31 out of 622 real vulnerabilities were hidden, a dangerous-suppression rate of 5.0%.

Performance differs between the two settings. On synthetic single-file cases the pipeline errs rarely, while on real applications, where a verdict depends on code distributed across a repository, roughly one suppression in four is incorrect. Section IV-B analyzes the cases where the classification was wrong and examines the common sources of error.

The escalation agent fired on 44 OWASP findings (3.6%) and 194 RealVuln findings (26.1%). Since it triggers only on uncertain verdicts or on FP below high certainty, every escalated finding ending as TP is one the agent overturned. On OWASP it overturned 19, all real vulnerabilities, and correctly confirmed 23 suppressions. On RealVuln it overturned 152, of which 145 are real vulnerabilities and 7 traps, recovering 145 vulnerabilities for 7 retained traps.

B. Model Comparison

Fig. 2 compares all four models on both benchmarks. The ranking by F1 is identical in each case, led by Qwen 3.8 27B, with the three leading models spanning 0.09 F1 on OWASP and 0.11 on RealVuln. Parameter count does not predict quality, as the 80B model ranks last and the 27B model leads. What separates them is suppression recall, which spans 0.329 to 0.958 on OWASP while precision there varies by less than 0.01. Suppression correctness is effectively constant across models, while they vary in how many FPs they suppress.

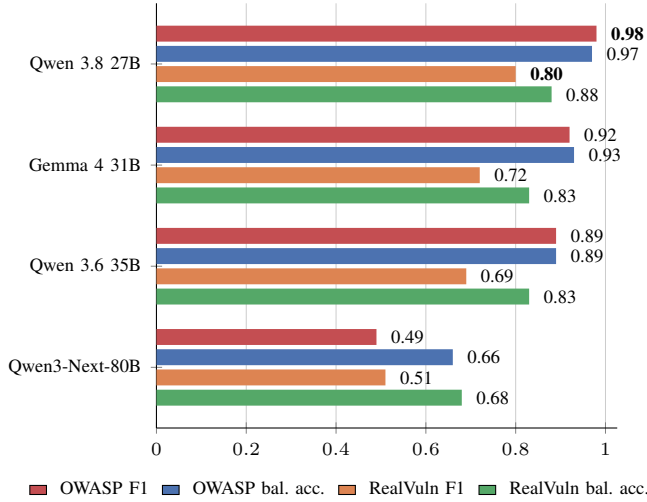


Fig. 2. F1-score and balanced accuracy of four models on both benchmarks, sorted by OWASP F1 with the best model on top. F1 refers to the FP class. The best F1-score per benchmark is highlighted.

The models differ in how aggressively they suppress, and this cost of aggressiveness differs between the two benchmarks. On OWASP recall is nearly free: Qwen 3.8 27B catches 95.8% of traps while hiding 2 of 452 real vulnerabilities. On RealVuln it is not, as recalls of 0.413 to 0.825 come with dangerous-suppression rates of 4.3% to 8.8%. The two do not move together strictly, however, since Qwen 3.8 27B reaches the highest recall at 5.0% while Qwen 3.6 35B hides 8.8% of the real vulnerabilities for a lower one. Qwen3-Next-80B marks the degenerate end, answering TP to almost everything. It therefore has the lowest dangerous-suppression rate on RealVuln, but leaves 522 of 778 OWASP traps in the report.

Cost varies more than quality does (see Table II), and escalation frequency drives it, since identical prompts give every model a similar input volume per call. Gemma 4 31B hands 38.1% of the RealVuln findings to the agent and needs 44% more tokens per finding than the leading model at a lower F1, whereas Qwen3-Next-80B never escalates on OWASP and is an order of magnitude faster at the worst F1. Token volume per finding otherwise tracks escalation rate rather than model size: Qwen3-Next-80B, which escalates least, is also cheapest in tokens on both benchmarks, while Gemma 4 31B and Qwen 3.6 35B, which escalate 4 to 9 times as often on RealVuln as on OWASP, need 2.3 to 3.2 times as many tokens per finding there. Classification is sequential, so per-finding latency is also throughput: the Qwen 3.8 27B run took 17.5 hours on OWASP and 19.5 on RealVuln.

C. What the Models Get Wrong

Of the 27 OWASP traps the model failed to suppress, 15 sit behind an incomplete guard at the sink, which the model rejects as bypassable. Six involve list operations after which the flagged variable in fact holds a fixed value, but the model traces the operation incorrectly and treats the value as attacker-

TABLE II
ESCALATION RATE, MEAN PER-FINDING LATENCY IN SECONDS, AND MEAN PER-FINDING TOKEN VOLUME OF THE FIRST-STAGE AND ESCALATION CALLS COMBINED, MEASURED OVER ALL CLASSIFIED FINDINGS OF EACH RUN.

Model	OWASP			RealVuln		
	Esc.	s	Tokens	Esc.	s	Tokens
Qwen 3.8 27B	3.6%	51.4	8,196	26.1%	94.8	21,669
Gemma 4 31B	4.1%	21.2	9,857	38.1%	37.0	31,114
Qwen 3.6 35B	4.4%	26.5	10,018	19.5%	33.0	22,719
Qwen3-Next-80B	0.0%	2.5	7,700	2.9%	4.9	9,182

controlled. The remaining six require information the snippet does not contain and the first stage did not escalate these findings. On RealVuln, 15 of the 20 surviving traps share one cause: the flagged line is safe, but the weakness the finding describes sits a few lines away in the same file, and the model reports the vulnerability it locates rather than the flagged one. This is intended behavior, as Polygraph judges the validity of SAST findings and is not designed to correct scanner output.

The 31 suppressed vulnerabilities divide less evenly. In 13 the rule’s CWE or line names a weakness the code at that point does not carry, such as MD5 password hashing reported as sensitive data exposure, and the model, like it was instructed to, judges the literal claim. In 9 findings the ground truth marks a risky pattern that no attacker-controlled data reaches and Polygraph therefore concludes as FP. In one finding the model’s counter-argument holds when it is checked against the source, so it is the label rather than the verdict that is wrong. That leaves 8 genuine misses, where the model settled the flagged construct and overlooked a fact one step away from it, such as a debug-setting that turns a deliberately raised exception into a full settings dump. Seven of the eight carried high certainty from the first stage and thus never reached the agent at all.

The dominant error is therefore a mismatch of questions. Polygraph asks whether the finding is correct at the specified location, while some labels are imprecise regarding their location and description. This is a limitation of the evaluation rather than of the system. Polygraph is built to judge the validity of SAST findings, not to correct their locations or descriptions.

V. CONCLUSION AND FUTURE WORK

We presented Polygraph, a tool that classifies SAST findings as TPs or FPs from a compact, finding-specific context. It reads and writes standard SARIF and records its verdicts in the standard `suppressions` field, so it operates between any compliant scanner and any compliant viewer. Its two-stage design keeps the common case cheap, as only the findings the static context does not settle escalate to an autonomous agent that explores the repository on its own.

On the OWASP Benchmark for Python, the best of four open-weight models reached an F1-score of 0.977 on the FP class, suppressing 95.8% of the traps at a dangerous-suppression rate of 0.4%. On RealVuln, whose verdicts depend

on code spread across a repository, it reached an F1-score of 0.797 and suppressed 82.5% of the traps at a dangerous-suppression rate of 5.0%. Escalation remained rare where the static context sufficed, firing on 3.6% of the OWASP findings, and recovered 145 real vulnerabilities on RealVuln for 7 retained traps. Parameter count did not predict quality, as the 27B model led and the 80B model ranked last.

Several directions remain open. All runs ran without data-flow analysis, since it is the slowest step of the pipeline, so the contribution of this context signal is still unmeasured. Selecting the enrichers per finding, guided by its CWE class, would limit that cost to the findings whose verdict depends on it. Second, the evaluation covers Python alone and consumes synthesized reports rather than live scanner output, so a broader study across languages, scanners, and production code is needed to show how far these results transfer. Finally, given that model size did not decide quality, post-training a small open-weight model with confirmed real-world TPs and FPs could result in better results overall.

REFERENCES

- [1] M. Vero, N. Mündler, V. Chibotaru, V. Raychev, M. Baader, N. Jovanović, J. He, and M. Vechev, “Baxbench: Can llms generate correct and secure backends?” 2025.
- [2] A. Nash, “The false positive tax: How bad automation destroys security program credibility,” Netcraft Blog, Sep. 2025. [Online]. Available: <https://www.netcraft.com/blog/the-false-positive-tax-how-bad-automation-destroys-security-program-credibility>
- [3] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, “Why don’t software developers use static analysis tools to find bugs?” in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 672–681.
- [4] OWASP Foundation, “OWASP benchmark,” OWASP Project, 2024. [Online]. Available: <https://owasp.org/www-project-benchmark/>
- [5] J. Pellaw and F. Raza, “Realvuln: Benchmarking rule-based, general-purpose llm, and security-specialized scanners on real-world code,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.13764>
- [6] B. Chess and G. McGraw, “Static analysis for security,” *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.
- [7] M. C. Fanning and L. J. Golding, “Static analysis results interchange format (SARIF) version 2.1.0,” OASIS Standard, mar 2020. [Online]. Available: <https://docs.oasis-open.org/sarif/sarif/v2.1.0/os/sarif-v2.1.0-os.html>
- [8] O. Tripp, M. Pistoia, S. J. Fink, M. Sridharan, and O. Weisman, “Taj: effective taint analysis of web applications,” *SIGPLAN Not.*, vol. 44, no. 6, p. 87–97, Jun. 2009. [Online]. Available: <https://doi.org/10.1145/1543135.1542486>
- [9] GitHub, “Codeql,” GitHub. [Online]. Available: <https://codeql.github.com/>
- [10] M. Iranmanesh, S. M. Sabet, S. Marefat, A. J. Ghasr, A. Wilson, I. Sharafaldin, and M. A. Tayebi, “Zerofalse: Improving precision in static analysis with llms,” *CoRR*, vol. abs/2510.02534, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.02534>
- [11] M. R. Ameen, M. T. U. Alam, and A. Islam, “Qasecclaw: A multi-agent LLM approach for false positive reduction in static application security testing,” *CoRR*, vol. abs/2605.01885, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2605.01885>
- [12] Y. Xiong and T. Zhang, “Sifting the noise: A comparative study of llm agents in vulnerability false positive filtering,” 01 2026.
- [13] R. Khojah, M. Mohamad, L. Erlenhov, F. G. de Oliveira Neto, and P. Leitner, “Llm company policies and policy implications in software organizations,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.06718>